

UD 9 - PRÁCTICA EVALUABLE 1



ÍNDICE

1. Introducción	3
2. Identificación del Equipo	3
3. Distribución de Roles	4
4. Diagrama Entidad-Relación (E-R)	5
5. Diagrama de Clases	8
Capa de Modelo	8
Relaciones entre clases	9
Anotaciones JPA más relevantes	9
6. Justificación de Tecnologías	11
7. Estrategia de Git	14
8. Estructura del Proyecto	15

1. Introducción

Para esta práctica de la UD 9, hemos desarrollado un sistema de gestión de contenidos de streaming usando Java. La idea principal era ir un paso más allá de lo convencional, utilizando ObjectDB para conectar nuestra aplicación con una base de datos orientada a objetos de forma sencilla y eficiente gracias a JPA. Durante el proceso, nos hemos enfrentado al reto de gestionar datos complejos (como usuarios, series y valoraciones) y asegurar que todo encaje perfectamente. Todo el desarrollo lo hemos coordinado a través de Git, lo que nos ha permitido trabajar de forma conjunta, repartirnos los roles y cuidar al máximo la calidad final del proyecto.

2. Identificación del Equipo

Nombre del proyecto: Proyecto práctica evaluable U9.

Asignatura: Programación.

Curso académico: 1ºDAM.

Nombre completo de todos los integrantes del equipo:

- Fátima Vázquez Rivera
- Alejandro Andrade Morales
- Juan Francisco Garrido Ariza
- Diego Manuel Carrasco
- Hugo Rodríguez Viguera

3. Distribución de Roles

Para garantizar una ejecución eficiente del software y asegurar la calidad del proyecto, el equipo ha implementado una distribución estratégica de roles.

Este enfoque permite que cada integrante se especialice en un área, garantizando que todos los requisitos técnicos y documentales sean cubiertos bajo una clara asignación de responsabilidades.

A continuación, se detalla la asignación de roles y la carga de trabajo equitativa establecida:

MIEMBRO DEL EQUIPO	ROL ASIGNADO	TAREAS PRINCIPALES	PORCENTAJE DE DEDICACIÓN
Fátima	Jefe De Proyecto	Coordina el equipo, gestiona los plazos, organiza las reuniones y supervisa el cumplimiento de los objetivos. Es el responsable último de la entrega.	20%
Diego	Diseñador De Base De Datos	Elabora el modelo conceptual (diagrama E-R), el modelo lógico y el diseño de las tablas. Define las restricciones de integridad, claves primarias y foráneas.	20%
Juan Francisco	Responsable de Calidad y Documentación	Redacta la memoria del proyecto, revisa la calidad del código, elabora los diagramas de clases y garantiza que el repositorio y el README están correctamente mantenidos.	20%
Hugo	Desarrollador de Interfaz	Diseña e implementa la interfaz de usuario (consola o gráfica) para interactuar con el sistema. Integra las funcionalidades CRUD en la interfaz.	20%
Alejandro	Diseñador Backend	Implementa la lógica de negocio en Java: clases de modelo, repositorios, servicios y la capa de acceso a la base de datos mediante JDBC.	20%

4. Diagrama Entidad-Relación (E-R)

El modelo Entidad-Relación es la herramienta fundamental para diseñar cualquier base de datos robusta. Su propósito es permitir visualizar la estructura lógica de la información y cómo interactúan los distintos elementos entre sí, antes de proceder a la implementación técnica.

A continuación, se detallan las entidades que componen el ecosistema de la plataforma y las reglas de negocio que rigen sus vínculos, clasificadas según su cardinalidad (la forma en que los registros de una tabla se relacionan con los de otra).

Entidades Principales:

- CONTENIDOS: Películas y series con metadatos técnicos.
- EPISODIOS: Componentes numerados que pertenecen a una serie.
- USUARIOS: Titulares de la cuenta con datos de contacto únicos.
- SUSCRIPCIÓN: El plan de pago activo del usuario.
- PERFILES: Subcuentas de visualización creadas por el usuario.
- REPRODUCCIONES: Registros individuales de visionado con progreso.
- VALORACIONES: Calificaciones y comentarios con identidad propia.
- CATEGORÍAS: Etiquetas temáticas de organización.

Relaciones Lógicas:

- Contiene: Una serie se compone de uno o varios episodios.
- Clasificado en: Un contenido se asocia a múltiples categorías y viceversa.
- Tiene: Un usuario mantiene vinculada su suscripción activa.
- Crea: Un usuario genera y gestiona uno o varios perfiles.
- Realiza: Un perfil genera el registro de una reproducción.
- Reproduce: Una reproducción identifica qué contenido específico se vio.
- Hace: Un perfil es el autor de una valoración.
- Valora: Una valoración vincula la opinión del perfil con un contenido.

Explicación de las Relaciones

Relaciones de Uno a Muchos (1:N)

- Serie → Episodios: Una serie se compone de muchos episodios, pero cada episodio pertenece exclusivamente a una única serie.
- Usuario → Perfiles: Un usuario puede crear y gestionar varios perfiles en su cuenta, pero cada perfil está vinculado a un solo usuario titular.
- Perfil → Reproducciones: Un perfil puede realizar múltiples reproducciones a lo largo del tiempo, pero cada registro de reproducción es único y pertenece a un solo perfil.
- Contenido → Reproducciones: Un mismo contenido (película o serie) puede ser reproducido muchas veces por diferentes perfiles, pero cada entrada de reproducción se refiere a un solo contenido específico.

- Perfil → Valoraciones: Un perfil puede dejar muchas valoraciones sobre distintos contenidos, pero cada valoración tiene un único perfil como autor.
- Contenido → Valoraciones: Un contenido puede recibir múltiples valoraciones de distintos usuarios, pero cada valoración está asociada a un solo contenido.

Relación de Muchos a Muchos (N:M)

- Contenidos ↔ Categorías: Un contenido (película o serie) puede estar clasificado en varias categorías temáticas (ej: Acción y Comedia), y una categoría puede agrupar a múltiples contenidos diferentes.

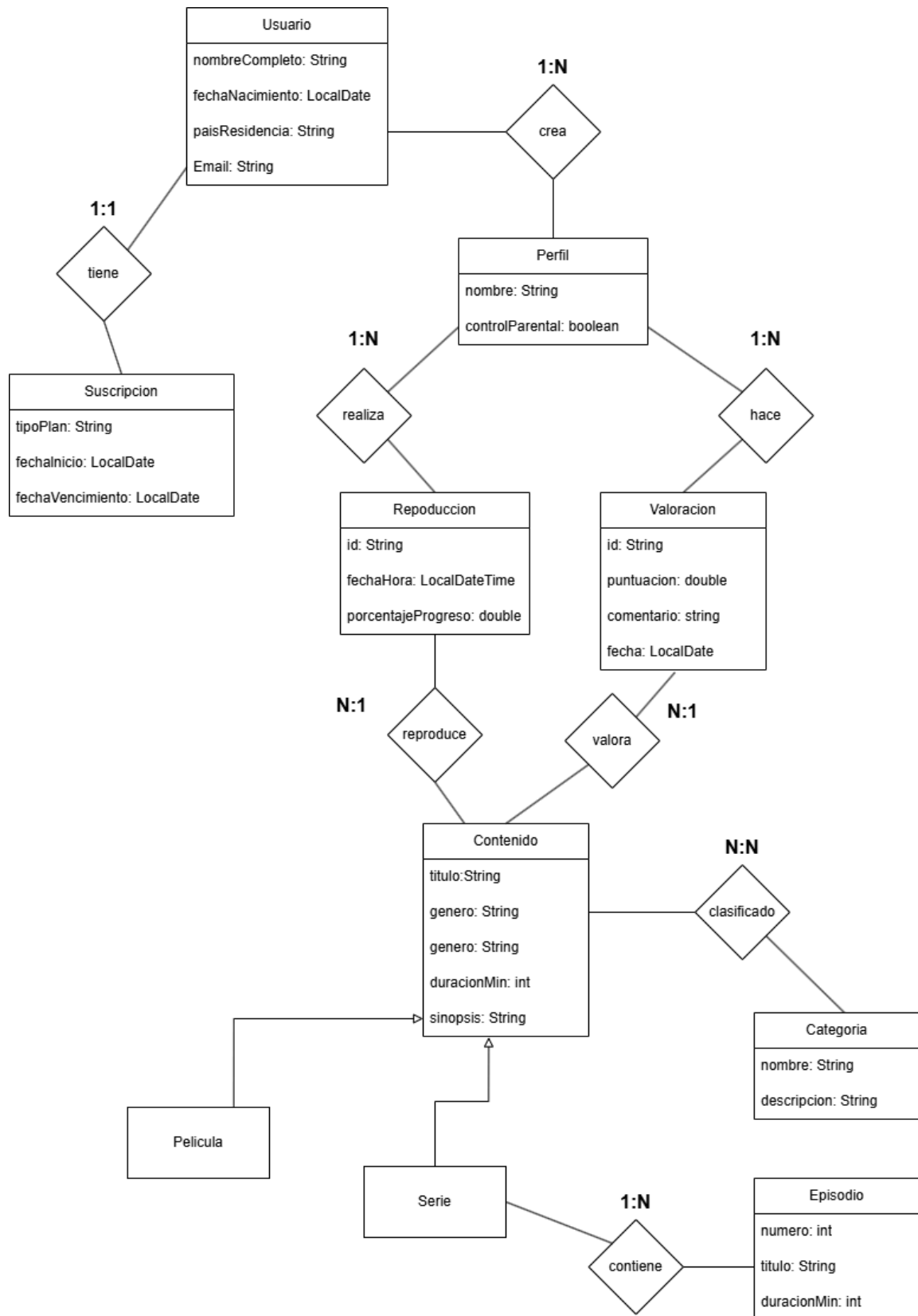
Relación de Uno a Uno (1:1)

- Usuario ↔ Suscripción: Según los requisitos, cada usuario dispone de una suscripción activa. Esto significa que un registro de usuario se vincula directamente con un único registro de suscripción que contiene su plan y vigencia.

Relación Asociativa (Entidades con identidad)

Reproducción y Valoración actúan como "puentes" entre los Perfiles y el Contenido:

- No son simples tablas de unión, sino entidades independientes porque guardan información propia (comentarios, puntuaciones y fechas) y poseen un identificador único que no depende de las otras tablas.



5. Diagrama de Clases

El diagrama UML de clases representa la estructura completa del proyecto "Streaming", mostrando cómo se organizan los objetos, qué información almacena y cómo se relacionan entre sí para que el sistema funcione correctamente.

El sistema se organiza en clases que representan las entidades del mundo real de una plataforma de streaming, todas ellas anotadas con JPA para que ObjectDB pueda guardarlas y recuperarlas de la base de datos automáticamente.

Capa de Modelo

Es el núcleo del sistema. Contiene todas las clases que representan los datos del proyecto. Todas están anotadas con `@Entity`, lo que le indica a JPA que deben ser persistidas en la base de datos, y sus atributos están protegidos con visibilidad privada para garantizar la integridad de los datos, accediendo a ellos mediante métodos getters y setters.

Usuario

Es la clase central del sistema. Representa a cada persona registrada en la plataforma y almacena sus datos principales. Un usuario puede tener una suscripción activa y crear uno o varios perfiles.

Suscripción

Está asociada directamente a un Usuario con una relación de uno a uno. Guarda el tipo de plan contratado, la fecha de inicio y la fecha de vencimiento. Esta clase permite controlar si un usuario tiene acceso activo a la plataforma.

Perfil

Representa cada perfil de usuario dentro de una cuenta. Almacena el nombre del perfil y si tiene control parental activado. Un usuario puede crear varios perfiles, y cada perfil puede realizar reproducciones y hacer valoraciones de forma independiente.

Reproducción

Registra cada vez que un perfil reproduce un contenido. Guarda el identificador, la fecha y hora exacta de la reproducción y el porcentaje de progreso, lo que permite saber hasta dónde ha visto el usuario ese contenido.

Valoración

Representa la puntuación y el comentario que un perfil deja sobre un contenido. Almacena el id de la valoración, la puntuación numérica, el texto del comentario y la fecha en que se realizó.

Contenido

Es una clase abstracta, lo que significa que no se puede instanciar directamente sino que sirve de base para otras clases más específicas. Contiene los atributos comunes a todo el contenido de la plataforma. Al ser abstracta, obliga a que todo contenido sea una Película o una Serie, ya que heredan de Contenido, lo que significa que tienen todos sus atributos más los propios de cada tipo.

Episodio

Pertenece a una Serie y representa cada capítulo. Almacena el número de episodios, su título y su duración en minutos. La relación entre Serie y Episodio es de composición, es decir un episodio no puede existir sin su serie.

Categoría

Permite clasificar el contenido por categorías. Tiene un nombre y una descripción, y se relaciona con Contenido mediante una relación de muchos a muchos, es decir un contenido puede pertenecer a varias categorías y una categoría puede agrupar varios contenidos.

Relaciones entre clases

Herencia

La relación más destacada es la que existe entre Contenido, Película y Serie. Contenido es la clase padre abstracta y tanto Película como Serie heredan todos sus atributos y comportamientos. Esto evita duplicar código y permite tratar a películas y series de forma uniforme cuando sea necesario.

Composición

El diagrama presenta varias relaciones de composición, donde una clase forma parte de otra y no tiene sentido sin ella. Un Usuario tiene exactamente una Suscripción asociada, que depende completamente de él y no puede existir de forma independiente. Del mismo modo, un Usuario puede crear uno o varios Perfiles, que también dependen directamente del usuario que los generó.

Por otra parte, una Serie contiene uno o varios Episodios, que no tienen ningún sentido fuera de la serie a la que pertenecen, por lo que si se elimina una serie sus episodios desaparecen con ella.

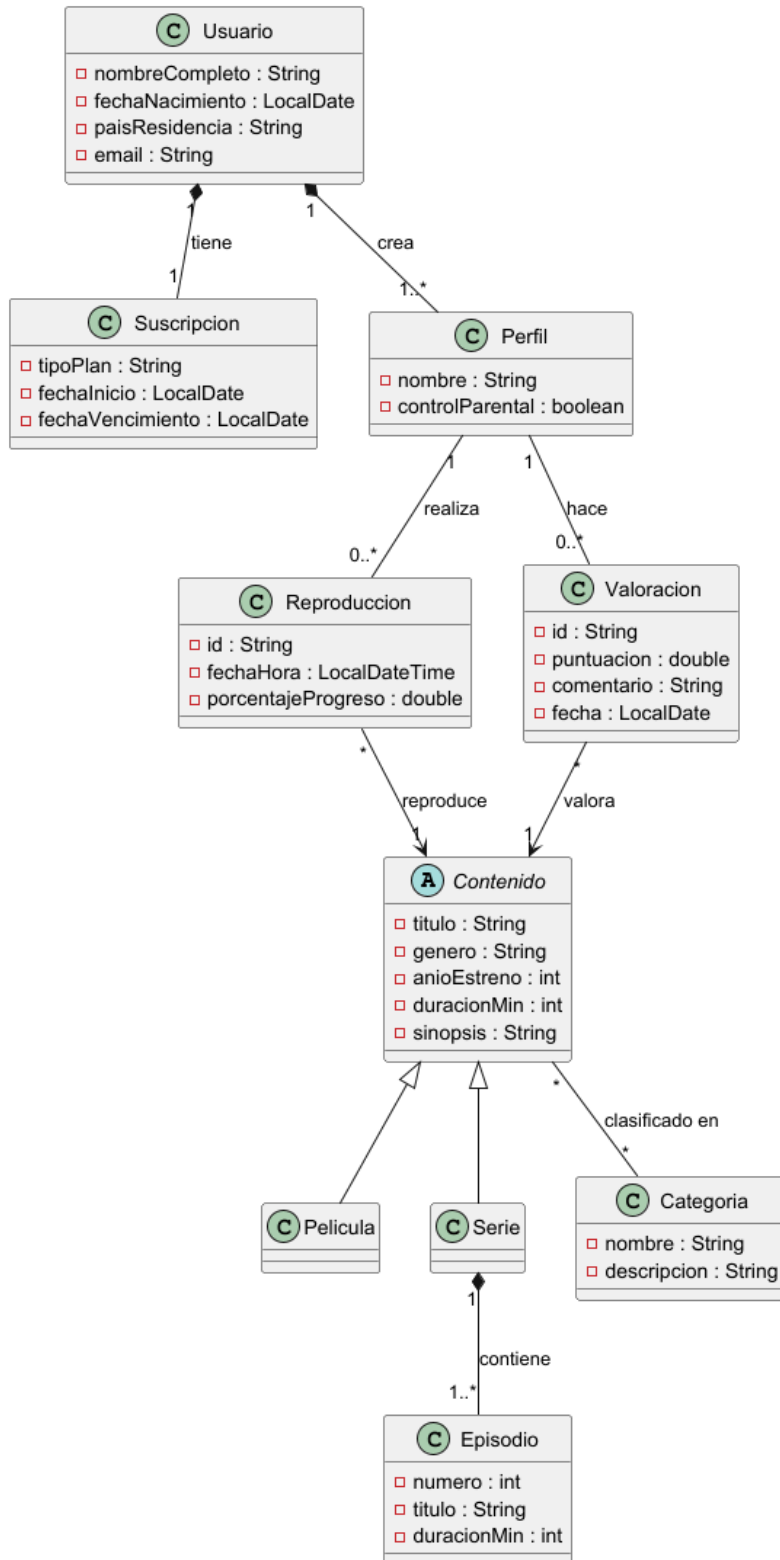
Anotaciones JPA más relevantes

Las anotaciones JPA son etiquetas que se colocan sobre las clases y atributos para indicarle a la base de datos cómo debe tratarlos.

Las más importantes en este proyecto son:

- @Entity para marcar una clase como persistente
- @Id para indicar cuál es el identificador único de cada objeto
- @OneToMany y @ManyToOne para las relaciones entre clases
- @ManyToMany para la relación entre Contenido y Categoría.

Gracias a estas anotaciones, ObjectDB sabe exactamente cómo guardar y recuperar cada objeto sin necesidad de escribir ninguna sentencia SQL.



6. Justificación de Tecnologías

La arquitectura tecnológica del proyecto "Streaming" ha sido seleccionada con el objetivo de maximizar la robustez, la mantenibilidad del código y la eficiencia en el trabajo colaborativo.

Cada herramienta ha sido elegida tras una fase de análisis comparativo, y priorizando la estabilidad del sistema.

Lenguaje de Programación: Java

Hemos optado por Java por ser un lenguaje de madurez tecnológica. Su máquina virtual de Java (JVM), garantiza la portabilidad del software (se escribe una vez, y se ejecuta en cualquier lugar) y su nivel de seguridad en tiempo de compilación que minimiza errores lógicos cruciales en una aplicación.

- **Ventajas experimentadas:**

La estructura orientada a objetos encajó perfectamente con la idea de representar cada entidad como una clase con sus datos y sus métodos.

- **Dificultades y soluciones:**

Al principio nos costó organizar bien los paquetes y decidir qué responsabilidad tenía cada clase. Lo fuimos resolviendo a medida que avanzaba el proyecto y veíamos cómo encajaban las piezas entre sí.

ObjectDB y JPA

Una de las decisiones más relevantes del proyecto fue la elección del sistema de persistencia de datos. Para ello comparamos dos enfoques principales: las bases de datos relacionales (como MySQL o PostgreSQL) y las bases de datos orientadas a objetos (como ObjectDB).

Bases de datos relacionales almacenan la información en tablas con filas y columnas, y utiliza SQL para consultarla. Son el estándar más extendido y funcionan muy bien cuando los datos tienen una estructura clara y bien definida. Sin embargo, cuando se trabaja con lenguajes orientados a objetos como Java, no se traducen de forma directa a tablas, lo que obliga a escribir código adicional y complica el mantenimiento.

ObjectDB, en cambio, es una base de datos orientada a objetos que almacena directamente los objetos Java tal y como están definidos en el código, sin necesidad de transformarlos.

Esto, permite trabajar de forma mucho más natural cuando el modelo de datos está definido en clases Java.

La razón por la que utilizamos y combinamos ObjectDB con JPA (Java Persistence API) es que JPA es el estándar de persistencia de Java, lo que significa que el código de acceso a datos no depende directamente de ObjectDB sino de una interfaz estándar, por lo que hace el proyecto más portable y mantenible.

- **Ventajas experimentadas:**

No tuvimos que escribir sentencias SQL ni preocuparnos por crear tablas manualmente. Bastaba con anotar las clases con @Entity y JPA se encargaba por lo que aceleró mucho el desarrollo del proyecto..

- **Dificultades y soluciones:**

La configuración inicial del archivo persistence.xml y entender cómo conectar ObjectDB con el proyecto Maven nos llevó tiempo. Lo resolvimos repasando la teoría de la unidad, lo que nos permitió entender mejor cómo debía estructurarse el archivo de configuración y qué parámetros eran necesarios para que la conexión con ObjectDB funcionara correctamente.

Entorno de desarrollo (IDE):

Elegimos IntelliJ IDEA porque es el IDE que usamos durante todo el curso y con el que más cómodos nos sentimos. Al ser el estándar en el módulo, todos los miembros del equipo partían del mismo entorno sin necesidad de adaptarse a nada nuevo.

- **Ventajas experimentadas:**

IntelliJ nos avisaba de los errores antes de ejecutar el programa, lo que ahorra mucho tiempo. También nos facilitó navegar entre clases, renombrar métodos en todos los sitios a la vez y ver la base de datos sin salir del IDE.

- **Dificultades y soluciones:**

En algunos ordenadores el proyecto no reconocía la versión de Java al importarlo y daba errores raros. Se solucionaba yendo a File > Project Structure y seleccionando el JDK correcto manualmente.

Herramientas adicionales: Maven

Para organizar nuestro proyecto y manejar las librerías necesarias, utilizamos Apache Maven. En el desarrollo de software, lo habitual es necesitar archivos externos (librerías) para que el código funcione. Antes, esto se hacía bajando archivos .jar de Internet y metiéndolos en carpetas a mano, lo cual es muy desordenado y da muchos errores. Lo usamos para dejar de gestionar las librerías manualmente. Con Maven, solo tenemos que escribir qué librerías necesitamos en un archivo llamado 'pom.xml', y la herramienta se encarga de descargarlas y conectarlas al proyecto automáticamente. Su implementación aseguró la estabilidad del proyecto al eliminar conflictos de versiones, garantizando que todos los desarrolladores trabajarán bajo una configuración de librerías idéntica (mismas dependencias en el archivo de configuración establecidas en Git).

- **Ventajas experimentadas:**

La estructura de carpetas que impone Maven (src/main/java, etc.) nos ayudó a tener el proyecto ordenado desde el principio y a saber siempre dónde estaba cada cosa.

- **Dificultades y soluciones:**

Cuando añadimos mal una dependencia el proyecto entero dejaba de compilar y no sabíamos por dónde empezar. Lo fuimos resolviendo leyendo la documentación y buscando información.

Herramientas adicionales: librerías externas, frameworks de interfaz gráfica, etc.

El proyecto hace uso de las siguientes librerías externas, todas declaradas como dependencias en el archivo pom.xml y gestionadas automáticamente por Maven.

ObjectDB (com.objectdb:objectdb:2.9.5)

ObjectDB es el motor de base de datos orientada a objetos que da soporte a toda la capa de persistencia del proyecto. Para que Maven pudiera descargarla fue necesario añadir el repositorio oficial de ObjectDB en el pom.xml (<https://m2.objectdb.com>), ya que esta librería no se encuentra en el repositorio central de Maven.

javax.persistence:javax.persistence-api:2.2

Es la especificación estándar de JPA (Java Persistence API) en su versión 2.2. Esta librería se define con las anotaciones (@Entity, @Id, @OneToMany, etc.) y las interfaces (EntityManager, EntityManagerFactory, Query) que el código utiliza para interactuar con la base de datos.

javax.transaction:javax.transaction-api:1.3

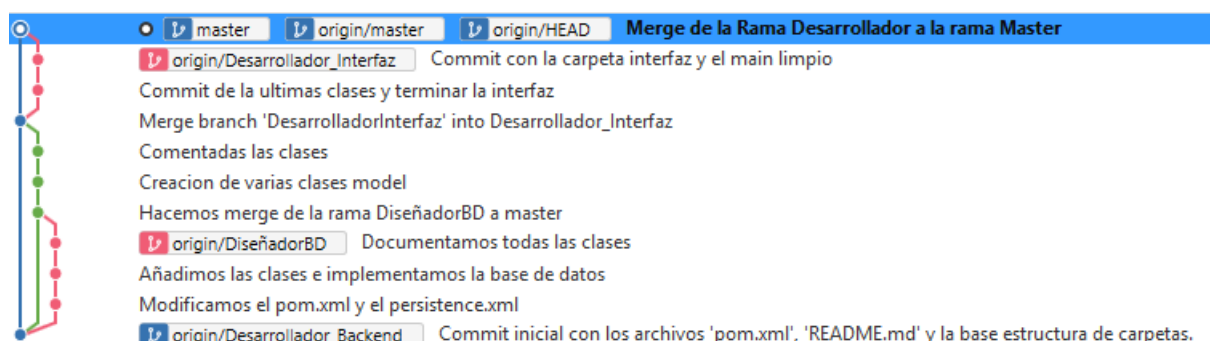
Es la API estándar de Java para la gestión de transacciones. Esta librería es necesaria porque JPA y ObjectDB la utilizan internamente para garantizar la consistencia de los datos.

- **Ventajas experimentadas:**

Al estar todas declaradas en el pom.xml, cualquier miembro del equipo puede clonar el repositorio y ejecutar el proyecto sin configurar nada manualmente, garantizando que todos trabajen con exactamente las mismas versiones.

- **Dificultades y soluciones:**

Al configurar un repositorio Maven adicional para ObjectDB supone un paso extra que puede complicar la inicialidad del proyecto.



8. Estructura del Proyecto

El software se organiza siguiendo una arquitectura que separa la lógica de presentación, el acceso a datos y el modelo de objetos. A continuación se detalla el árbol de directorios completo y una explicación brevemente de cada componente.

ESTRUCTURAS DE CARPETAS

PE9_STREAMING/

	.gitignore	# Archivos excluidos del control de versiones
	pom.xml	# Configuración Maven: dependencias y plugins
	README.md	# Documentación general del proyecto
	— .idea	# Configuración del entorno IntelliJ IDEA
	.gitignore	# Exclusiones de IntelliJ para Git
	encodings.xml	# Codificación de caracteres del proyecto
	misc.xml	# Versión del JDK y configuración técnica
	vcs.xml	
	— objectdb-2.9.5	# Librería ObjectDB incluida en el proyecto
	— .idea	# Config IntelliJ del módulo ObjectDB
	.gitignore	
	misc.xml	
	modules.xml	
	— libraries	# Distribución de ObjectDB
	bin.xml	
	doc.xml	
	jakarta.xml	
	src.xml	
	— objectdb-2.9.5	# Distribución de ObjectDB
	objectdb.conf	
	readme.html	
	— bin	# Ejecutables y JARs de ObjectDB
	enhancer.bat	
	enhancer.sh	
	explorer-2.9.5.exe	
	explorer-2.9.5.jar	
	explorer.sh	
	javax.persistence-api-2.2.jar	
	javax.transaction-api-1.3.jar	
	objectdb-2.9.5.jar	
	server-2.9.5.exe	
	server.sh	
	— jakarta	# Variantes Jakarta de los ejecutables
	enhancer-jk.bat	
	enhancer-jk.sh	
	explorer-jk-2.9.5.exe	
	explorer-jk-2.9.5.jar	
	explorer-jk.sh	
	jakarta.persistence-api-3.2.0.jar	
	objectdb-jk-2.9.5.jar	

	server-jk-2.9.5.exe	
	server-jk.sh	
	└─db	# Bases de datos de ejemplo ObjectDB
	clinica.odt	
	point.odt	
	world.odt	
	└─doc	# Documentación Javadoc de JPA
	jakarta.persistence-api-3.2.0-javadoc.jar	
	javax.persistence-api-2.2-javadoc.jar	
	ObjectDB 2.9.pdf	
	└─license	# Textos de licencias de las APIs
	jdo-apache-license-2.0.txt	
	jpa-2-eclipse-public-license-1.0.html	
	jpa-3-eclipse-public-license-2.0.html	
	jpa-eclipse-distribution-license-1.0.html	
	objectdb-license-2.0.4.html	
	└─src	# Código fuente de las APIs JPA
	jakarta.persistence-api-3.2.0-sources.jar	
	javax.persistence-api-2.2-sources.jar	
	└─jakarta.persistence-api-3.1.0-sources	# Fuentes de JPA 3.1
	module-info.java	
	└─jakarta	# Paquete raíz Jakarta
	└─persistence	# Anotaciones e interfaces JPA 3.1
	Access.java	
	AccessType.java	
	AssociationOverride.java	
	AssociationOverrides.java	
	AttributeConverter.java	
	AttributeNode.java	
	AttributeOverride.java	
	AttributeOverrides.java	
	Basic.java	
	Cache.java	
	Cacheable.java	
	CacheRetrieveMode.java	
	CacheStoreMode.java	
	CascadeType.java	
	CollectionTable.java	
	Column.java	
	ColumnResult.java	
	ConstraintMode.java	
	ConstructorResult.java	
	Convert.java	
	Converter.java	
	Converts.java	

			DiscriminatorColumn.java
			DiscriminatorType.java
			DiscriminatorValue.java
			ElementCollection.java
			Embeddable.java
			Embedded.java
			EmbeddedId.java
			Entity.java
			EntityExistsException.java
			EntityGraph.java
			EntityListeners.java
			EntityManager.java
			EntityManagerFactory.java
			EntityNotFoundException.java
			EntityResult.java
			EntityTransaction.java
			Enumerated.java
			EnumType.java
			ExcludeDefaultListeners.java
			ExcludeSuperclassListeners.java
			FetchType.java
			FieldResult.java
			FlushModeType.java
			ForeignKey.java
			GeneratedValue.java
			GenerationType.java
			Id.java
			IdClass.java
			Index.java
			Inheritance.java
			InheritanceType.java
			JoinColumn.java
			JoinColumns.java
			JoinTable.java
			Lob.java
			LockModeType.java
			LockTimeoutException.java
			ManyToMany.java
			ManyToOne.java
			MapKey.java
			MapKeyClass.java
			MapKeyColumn.java
			MapKeyEnumerated.java
			MapKeyJoinColumn.java
			MapKeyJoinColumns.java
			MapKeyTemporal.java
			MappedSuperclass.java
			MapsId.java
			NamedAttributeNode.java
			NamedEntityGraph.java
			NamedEntityGraphs.java

		NamedNativeQueries.java
		NamedNativeQuery.java
		NamedQueries.java
		NamedQuery.java
		NamedStoredProcedureQueries.java
		NamedStoredProcedureQuery.java
		NamedSubgraph.java
		NonUniqueResultException.java
		NoResultException.java
		OneToMany.java
		OneToOne.java
		OptimisticLockException.java
		OrderBy.java
		OrderColumn.java
		orm_2_2.xsd
		orm_3_0.xsd
		orm_3_1.xsd
		package-info.java
		Parameter.java
		ParameterMode.java
		Persistence.java
		PersistenceContext.java
		PersistenceContexts.java
		PersistenceContextType.java
		PersistenceException.java
		PersistenceProperty.java
		PersistenceUnit.java
		PersistenceUnits.java
		PersistenceUnitUtil.java
		PersistenceUtil.java
		persistence_2_2.xsd
		persistence_3_0.xsd
		PessimisticLockException.java
		PessimisticLockScope.java
		PostLoad.java
		PostPersist.java
		PostRemove.java
		PostUpdate.java
		PrePersist.java
		PreRemove.java
		PreUpdate.java
		PrimaryKeyJoinColumn.java
		PrimaryKeyJoinColumns.java
		Query.java
		QueryHint.java
		QueryTimeoutException.java
		RollbackException.java
		SecondaryTable.java
		SecondaryTables.java
		SequenceGenerator.java
		SequenceGenerators.java

		SharedCacheMode.java	
		SqlResultSetMapping.java	
		SqlResultSetMappings.java	
		StoredProcedureParameter.java	
		StoredProcedureQuery.java	
		Subgraph.java	
		SynchronizationType.java	
		Table.java	
		TableGenerator.java	
		TableGenerators.java	
		Temporal.java	
		TemporalType.java	
		TransactionRequiredException.java	
		Transient.java	
		Tuple.java	
		TupleElement.java	
		TypedQuery.java	
		UniqueConstraint.java	
		ValidationMode.java	
		Version.java	
		——criteria	# API de consultas tipadas
		AbstractQuery.java	
		Collection.Join.java	
		CommonAbstractCriteria.java	
		CompoundSelection.java	
		CriteriaBuilder.java	
		CriteriaDelete.java	
		CriteriaQuery.java	
		CriteriaUpdate.java	
		Expression.java	
		Fetch.java	
		FetchParent.java	
		From.java	
		Join.java	
		JoinType.java	
		ListJoin.java	
		MapJoin.java	
		Order.java	
		package-info.java	
		ParameterExpression.java	
		Path.java	
		PluralJoin.java	
		Predicate.java	
		Root.java	
		Selection.java	
		SetJoin.java	
		Subquery.java	
		——metamodel	# Metamodelo estático de entidades JPA
		Attribute.java	

			BasicType.java	
			Bindable.java	
			CollectionAttribute.java	
			EmbeddableType.java	
			EntityType.java	
			IdentifiableType.java	
			ListAttribute.java	
			ManagedType.java	
			MapAttribute.java	
			MappedSuperclassType.java	
			Metamodel.java	
			package-info.java	
			PluralAttribute.java	
			SetAttribute.java	
			SingularAttribute.java	
			StaticMetamodel.java	
			Type.java	
			└─spi	# Interfaces de proveedor JPA
			ClassTransformer.java	
			LoadState.java	
			package-info.java	
			PersistenceProvider.java	
			PersistenceProviderResolver.java	
			PersistenceProviderResolverHolder.java	
			PersistenceUnitInfo.java	
			PersistenceUnitTransactionType.java	
			ProviderUtil.java	
			TransformerException.java	
			└─META-INF	# Metadatos del JAR de fuentes
			LICENSE.md	# Licencia del proyecto Jakarta
			MANIFEST.MF	# Manifiesto del JAR
			NOTICE.md	# Avisos legales o noticias
			└─maven	# Metadatos Maven
			└─jakarta.persistence	
			└─jakarta.persistence-api	
			pom.properties	
			pom.xml	
			└─src	# Código fuente principal del proyecto streaming
			└─main	# Fuentes y recursos principales
			└─java	# Clases Java del proyecto
			Main.java	# Entrada de la aplicación
			└─Interfaz	# Presentación de interacción por consola con el usuario
			ContenidoInterfaz.java	
			UsuarioInterfaz.java	
			Utils.java	# Funciones auxiliares para la lectura de datos y formato en consola
			ValoracionInterfaz.java	

[illegible]